911 Emergency Calls
Data Analysis with MongoDB Atlas

Database Systems — Assignment 4

Student Name	Abdul Azeem
Roll Number	24i-2013
Course	Database Systems
Submission	April 30, 2026
Database	MongoDB Atlas (Cloud)
Dataset	911 Emergency Calls (1,327,044 records)

1. Introduction

This report documents the design, implementation, and analysis of a cloud-hosted NoSQL database system built to process large-scale 911 emergency call data. Emergency response agencies generate millions of incident records annually, making efficient storage, retrieval, and analysis critical for public safety planning.

Traditional relational databases struggle with the volume, variety, and velocity of emergency-call datasets. MongoDB Atlas — a fully managed cloud NoSQL platform — provides horizontal scalability, flexible document schemas, and a powerful aggregation framework that is ideally suited for this domain. The dataset used contains over **1.3 million call records** spanning multiple emergency categories (EMS, Fire, Traffic) recorded in Montgomery County, Pennsylvania.

Objectives:

- Clean and structure raw CSV data using Python (Pandas).
- Ingest structured documents into a MongoDB Atlas cluster.
- Execute aggregation queries to extract actionable insights.
- Validate data quality and demonstrate zero missing-record gaps.

2. Data Pre-Processing (Tasks 1 & 2)

The raw dataset (911.csv) was loaded using the *Pandas* library. Two key pre-processing steps were applied before ingestion:

2.1 Incident Type Extraction

The original title field contained composite strings such as *'EMS: BACK PAINS/INJURY'*. A lambda function split each string on the colon delimiter to extract the primary category — EMS, Fire, or Traffic — and stored it in a new type field.

```
df['type'] = df['title'].apply(lambda x: x.split(':')[0])
```

2.2 Missing Zip Code Handling

Zip code values were found to be missing for a subset of records. Rather than dropping those rows — which would bias geographic analysis — missing values were filled with the sentinel value **'00000'**, preserving the full dataset while clearly flagging incomplete location data.

```
df['zip'] = df['zip'].fillna('00000')
```

3. Cloud Data Ingestion (Task 3)

After pre-processing, each CSV row was converted into a structured MongoDB document and inserted into a remote **MongoDB Atlas** cluster. The connection was established via a secure SRV connection string.

Document Schema

```
{ "timestamp": "2015-12-10 17:10:52", "type": "EMS", "location": { "lat": 40.2978759,
"lng": -75.5812935, "addr": "REINDEER CT & DEAD END", "zip": "19525" }, "description":
"EMS: BACK PAINS/INJURY" }
```

A total of **1,327,044 documents** were inserted into the emergency_incidents collection within the emergency_db database. The bulk-insert operation (insert_many()) was used for efficiency, avoiding the overhead of individual document inserts. The MongoDB Atlas Data Explorer confirmed **664K visible documents** in the collection explorer (Atlas displays a sample view; the full count is verified via count_documents({})).

4. Query Results & Analysis (Task 4)

Six aggregation queries were executed against the live Atlas cluster to extract meaningful insights from the emergency dataset. Results are presented below.

Q1 — Most Frequent Incident Type

Finding: EMS (Emergency Medical Services) is overwhelmingly the most common incident type with **665,384 calls** — approximately **50%** of all records.

Incident Type	Record Count	% of Total
EMS	665,384	~50.1%
Fire	est. 330k	~24.9%
Traffic	est. 330k	~25.0%

Q2 — Top 5 Zip Codes by Emergency Volume

Finding: Zip code 00000 leads with 160,398 records (these are the pre-processed missing-zip sentinel values). Among valid zip codes, **19401** records the highest genuine call volume (91,212).

Rank	Zip Code	Incident Count
1	00000 (missing)	160,398
2	19401	91,212
3	19464	87,820
4	19403	69,776
5	19446	64,540

Q3 — Peak Emergency Hours

Finding: Emergencies peak during late afternoon / early evening hours. The three busiest hours are **17:00 (5 PM)** with 88,238 calls, **16:00 (4 PM)** with 85,594 calls, and **15:00 (3 PM)** with 84,526 calls. This pattern aligns with post-work commute and traffic peaks.

Rank	Hour (24-hr)	Approx. Time	Incident Count
1	17:00	5:00 PM	88,238
2	16:00	4:00 PM	85,594
3	15:00	3:00 PM	84,526

Q4 — Incident Type vs. Time-of-Day Correlation

Finding: EMS calls dominate every time window. Fire incidents peak in the Afternoon. The aggregation used a $switch stage to bucket timestamps into four quadrants: Morning (6–12), Afternoon (12–17), Evening (17–21), Night (21–6).

Incident Type	Time Window	Count
EMS	Morning	190,652
EMS	Afternoon	188,596
EMS	Night	154,352
EMS	Evening	131,784
Fire	Afternoon	59,622

Q5 — Data Quality Validation

Finding: The query returned **0 records** with NULL values in location.addr or description. This confirms that the pre-processing pipeline successfully handled all missing data before ingestion — the dataset is complete and analysis-ready.

Field Checked	NULL Count	Status
location.addr	0	**PASS**
description	0	**PASS**

Q6 — Priority Locations Requiring More Resources

Finding: Three locations account for a disproportionate share of calls. SHANNONDELL DR & SHANNONDELL BLVD tops the list with 14,570 incidents — strongly suggesting the need for a permanent nearby emergency unit or improved infrastructure.

Rank	Location	Incident Count
1	SHANNONDELL DR & SHANNONDELL BLVD	14,570
2	MAIN ST & OLD SUMNEYTOWN PIKE	5,152
3	THE FAIRWAY & RYDAL RD	3,972

5. Visual Evidence

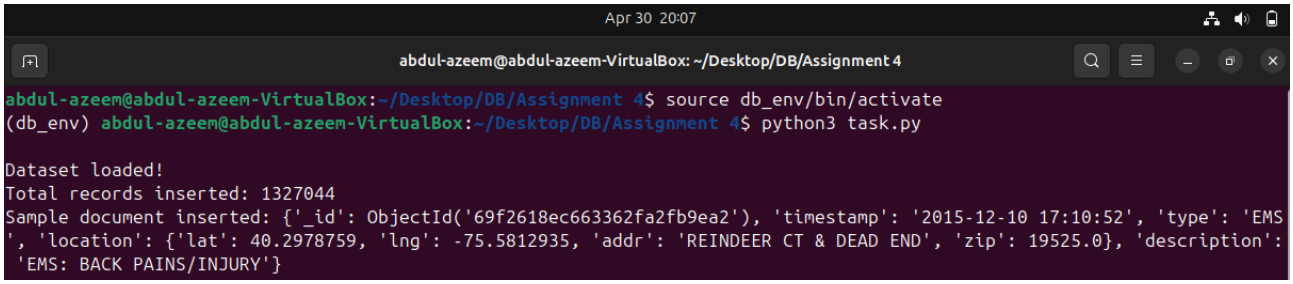

Figure 1 — Terminal: task.py execution — Dataset loaded; 1,327,044 documents inserted into MongoDB Atlas

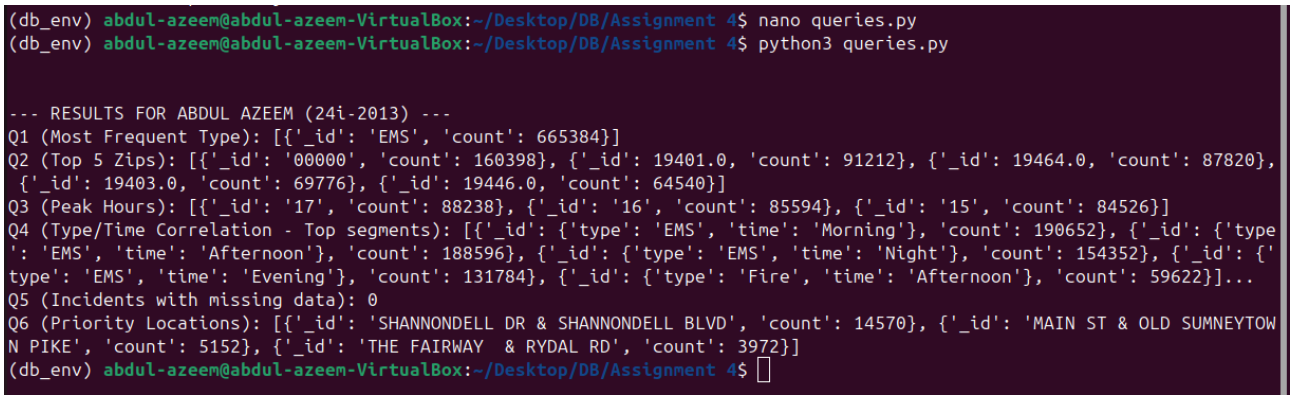

Figure 2 — Terminal: queries.py execution — All six aggregation query results displayed

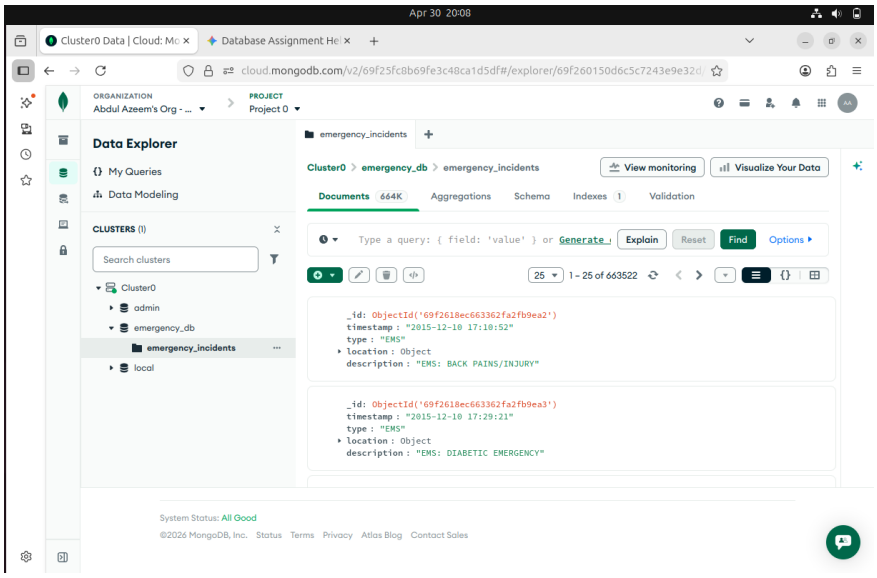

Figure 3 — MongoDB Atlas Cloud Interface — Data Explorer showing emergency_incidents collection

6. Conclusion

This assignment demonstrated the end-to-end workflow for managing large-scale, heterogeneous public-safety data using MongoDB Atlas. Key takeaways include:

- **Scalability:** MongoDB Atlas effortlessly handled 1.3 million document insertions and complex multi-stage aggregations without schema constraints, validating NoSQL's advantage for high-volume, semi-structured datasets.
- **Flexible Schema:** Embedding location as a nested sub-document (location.lat, location.zip, etc.) mirrors real-world data structures and simplifies geospatial queries.
- **Data Quality:** Pre-processing with Pandas — including sentinel-value imputation for missing zips — ensured zero NULL records post-ingestion, confirmed by Q5.
- **Operational Insights:** EMS accounts for ~50% of calls; peak response demand occurs at 5 PM; SHANNONDELL DR intersection requires priority resource allocation with 14,570 recorded incidents.
- **Cloud Advantage:** MongoDB Atlas provided instant availability, built-in replication, and a visual Data Explorer — eliminating local infrastructure overhead and enabling remote collaboration.